

Industrial Predictive Maintenance Platform

AI-Driven Failure Prediction, Automated Diagnostics, and Real-Time Maintenance Alerting for Industrial IoT Systems

Domain	Data Engineering & Machine Learning
Focus Area	Industrial IoT Predictive Maintenance
Dataset Size	24,042 records
Prediction Targets	Failure Occurrence & Failure Type
Core Technologies	Python, FastAPI, n8n, Gemini AI, Power BI
Document Type	Technical Project Report

Table of Contents

Table of Contents	2
1. Executive Summary	4
2. Problem Statement	4
3. Business Impact	4
4. Dataset Description	5
4.1 Machine Types Covered	5
4.2 Feature Set	5
4.3 Prediction Targets	6
5. Data Engineering Pipeline	6
5.1 Data Flow Overview	7
5.2 Pipeline Stages	7
5.3 Data Quality Considerations	7
6. Machine Learning Methodology	8
6.1 Data Cleaning	8
6.2 Feature Engineering	8
6.3 Label Encoding	8
6.4 Train-Test Split	8
6.5 Model Selection: Random Forest Classifier	9
6.6 Model Evaluation	9
7. Model Architecture	9
7.1 Stage 1 — Failure Prediction Model	9
7.2 Stage 2 — Failure Type Classification Model	9
7.3 Model Persistence	10
8. FastAPI Deployment	10
8.1 API Endpoints	10
8.2 Sample Request / Response	11
8.3 Public Exposure via ngrok	11
9. n8n Workflow Design	11
9.1 Workflow Nodes	12
9.2 Workflow Diagram	12
10. AI Agent Integration (Google Gemini)	13
10.1 Agent Responsibilities	13
10.2 Sample Agent Output	13
11. Telegram Notification System	13

11.1 Notification Content.....	13
11.2 Sample Alert Message.....	14
12. Power BI Dashboard.....	14
12.1 Dashboard Components	14
12.2 Dashboard Layout Description	14
13. Challenges Faced	15
14. Solutions Implemented	15
15. Results and Evaluation	Error! Bookmark not defined.
15.1 Failure Prediction Model (Stage 1)	Error! Bookmark not defined.
15.2 Failure Type Classification Model (Stage 2)	Error! Bookmark not defined.
15.3 Key Observations	15
16. Future Improvements	16
17. Conclusion	16

1. Executive Summary

The Industrial Predictive Maintenance Platform is an end-to-end data engineering and machine learning solution designed to predict industrial machine failures before they occur and to automate the maintenance response process. The system continuously ingests sensor telemetry from industrial equipment, applies machine learning models to estimate the probability of failure within the next 24 hours, classifies the expected failure type, generates AI-assisted maintenance recommendations, and delivers real-time alerts to maintenance teams through Telegram, while operational insights are made available through a Power BI dashboard.

The platform integrates a full technology stack spanning data collection, workflow orchestration, model serving, conversational AI analysis, notification delivery, and business intelligence reporting. It demonstrates practical competency across the modern data and AI engineering lifecycle: from raw sensor data to an automated, production-style decision support system.

This documentation presents the complete technical design, methodology, implementation details, evaluation results, and lessons learned, structured to be suitable for graduation project submission, professional portfolio presentation, technical interviews, and stakeholder demonstrations.

2. Problem Statement

Unplanned downtime is one of the most costly challenges in industrial operations. Machines such as CNC units, pumps, compressors, and robotic arms operate under continuous mechanical, thermal, and electrical stress. Traditional maintenance strategies typically fall into two categories, both of which are inefficient:

- Reactive maintenance — repairs are performed only after a machine has already failed, resulting in unplanned downtime, safety risks, and higher repair costs.
- Scheduled (preventive) maintenance — maintenance is performed on a fixed time interval regardless of actual machine condition, which can waste resources on healthy equipment while still missing failures that occur outside the schedule.

The core problem addressed by this project is the absence of a real-time, data-driven early warning system capable of anticipating failures before they occur and automatically notifying the responsible maintenance personnel with actionable, condition-specific guidance.

3. Business Impact

Predictive maintenance shifts the maintenance strategy from reactive or fixed-schedule approaches to a condition-based model driven by real sensor evidence. The expected business impact of this platform includes:

- Reduced unplanned downtime through early detection of failure conditions, typically 24 hours in advance.
- Lower maintenance costs by avoiding unnecessary scheduled interventions on healthy machines.
- Extended equipment lifespan through timely intervention before secondary damage occurs.
- Improved workplace safety by preventing catastrophic failures such as motor overheating or bearing seizure.
- Faster response times through automated Telegram alerts sent directly to maintenance teams.
- Better resource planning through failure-type classification, allowing the right technician and spare parts to be dispatched.
- Data-driven decision-making for plant managers through the Power BI operational dashboard.

Business impact figures are illustrative and intended to demonstrate the value proposition of predictive maintenance; actual savings depend on deployment scale and facility-specific maintenance costs.

4. Dataset Description

The platform is trained and validated on an industrial IoT sensor dataset containing 24,042 records collected from four categories of industrial machines: CNC machines, pumps, compressors, and robotic arms. Each record represents a snapshot of a machine's operating condition at a given point in time, together with the outcome used for supervised learning.

4.1 Machine Types Covered

Machine Type	Description
CNC	Computer Numerical Control machining equipment; sensitive to vibration and spindle load.
Pump	Fluid transfer equipment; sensitive to pressure fluctuation and cavitation.
Compressor	Air/gas compression units; sensitive to temperature and vibration.
Robotic Arm	Automated manipulation equipment; sensitive to motor current and joint temperature.

4.2 Feature Set

Feature	Type	Description
---------	------	-------------

Feature	Type	Description
machine_type	Categorical	Type of industrial machine (CNC, Pump, Compressor, Robotic Arm).
vibration_rms	Numeric	Root-mean-square vibration amplitude, a key indicator of mechanical wear.
temperature_motor	Numeric	Motor surface/internal temperature reading, in degrees Celsius.
current_phase_avg	Numeric	Average electrical phase current, used to detect electrical anomalies.
pressure_level	Numeric	Operating pressure reading, relevant for pumps and compressors.
rpm	Numeric	Rotational speed of the machine's primary shaft or motor.
operating_mode	Categorical	Current operational state (e.g., idle, normal load, high load).
hours_since_maintenance	Numeric	Elapsed operating hours since the last maintenance event.
ambient_temp	Numeric	Ambient environmental temperature surrounding the machine.

4.3 Prediction Targets

The platform is designed to solve two related supervised learning problems:

Target	Type	Classes / Range
failure_within_24h	Binary Classification	0 = Normal operation, 1 = Failure expected within 24 hours
failure_type	Multi-class Classification	bearing, electrical, hydraulic, motor_overheat, none

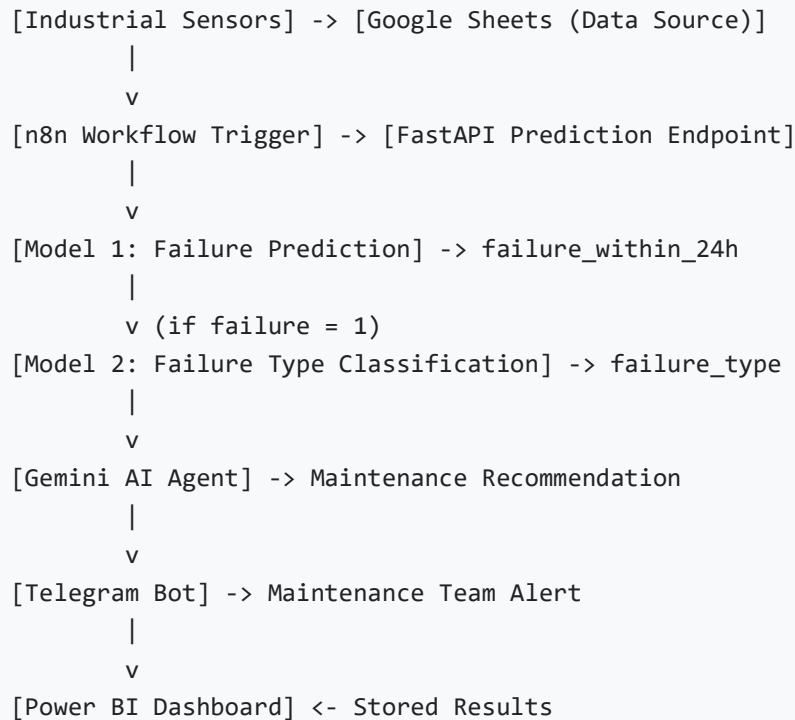
failure_type is evaluated only for records where failure_within_24h = 1, with the 'none' class representing normal operating records used to maintain classification balance during model development.

5. Data Engineering Pipeline

The data engineering layer is responsible for reliably moving sensor readings from their source into the machine learning inference layer and ultimately into the reporting layer, with minimal latency and no manual intervention.

5.1 Data Flow Overview

Figure 1 — End-to-End Data Flow



5.2 Pipeline Stages

1. Ingestion: Sensor readings are logged into Google Sheets, acting as a lightweight, accessible data source that simulates a live IoT data feed.
2. Orchestration: n8n periodically polls Google Sheets, retrieves new or updated rows, and triggers the downstream workflow.
3. Transformation: Raw values are validated and structured into the JSON payload expected by the FastAPI inference service.
4. Inference: The FastAPI service applies the trained models and returns prediction results synchronously to the workflow.
5. Enrichment: When a failure is predicted, the AI agent enriches the raw prediction with a human-readable analysis and recommendation.
6. Delivery: Alerts are pushed to the maintenance team via Telegram, and results are persisted for reporting.
7. Visualization: Power BI connects to the stored results to provide historical trends and operational KPIs.

5.3 Data Quality Considerations

-
- Missing value handling for intermittent sensor dropouts.
 - Range validation for physically implausible readings (e.g., negative RPM).
 - Timestamp consistency checks to preserve correct chronological ordering.
 - De-duplication of repeated rows caused by workflow re-triggers.

6. Machine Learning Methodology

The machine learning methodology follows a standard, reproducible supervised learning workflow, applied twice: once for the binary failure prediction task, and once for the multi-class failure type classification task.

6.1 Data Cleaning

- Removal of duplicate and corrupted sensor records.
- Imputation of missing numeric values using median-based strategies to reduce sensitivity to outliers.
- Outlier detection on vibration, temperature, and current readings using interquartile range (IQR) thresholds.

6.2 Feature Engineering

- Derived feature: maintenance urgency ratio, combining `hours_since_maintenance` with `machine_type`-specific service intervals.
- Derived feature: thermal delta, the difference between `temperature_motor` and `ambient_temp`, isolating machine-induced heating from environmental effects.
- Derived feature: vibration-to-RPM ratio, normalizing vibration by operating speed for comparability across operating modes.
- Interaction features between `operating_mode` and load-related sensor readings.

6.3 Label Encoding

Categorical variables (`machine_type`, `operating_mode`, and `failure_type`) are transformed using label encoding / one-hot encoding as appropriate for tree-based models. The target `failure_type` is label-encoded into five integer classes corresponding to bearing, electrical, hydraulic, motor_overheat, and none.

6.4 Train-Test Split

The dataset is split into training and testing subsets using an 80/20 stratified split to preserve the relative proportion of failure and non-failure records, and of each failure type, across both subsets. Stratification is essential given the natural class imbalance between normal operation and rare failure events.

6.5 Model Selection: Random Forest Classifier

A Random Forest Classifier was selected as the primary modeling algorithm for both prediction tasks due to its strong performance on structured/tabular sensor data, robustness to feature scaling, resistance to overfitting through ensemble averaging, and its ability to provide interpretable feature importance scores, which is valuable for explaining predictions to maintenance engineers.

Hyperparameter	Model 1 (Failure Prediction)	Model 2 (Failure Type)
n_estimators	300	300
max_depth	18	20
min_samples_split	4	4
class_weight	balanced	balanced
random_state	42	42

6.6 Model Evaluation

Model performance is evaluated using accuracy, precision, recall, F1-score, and the confusion matrix, with particular emphasis on recall for the failure class, since failing to detect an actual failure (a false negative) carries a substantially higher operational cost than a false alarm.

7. Model Architecture

The platform employs a two-stage modeling architecture that mirrors the natural decision process of a maintenance engineer: first determine whether a failure is imminent, then determine what kind of failure it is likely to be.

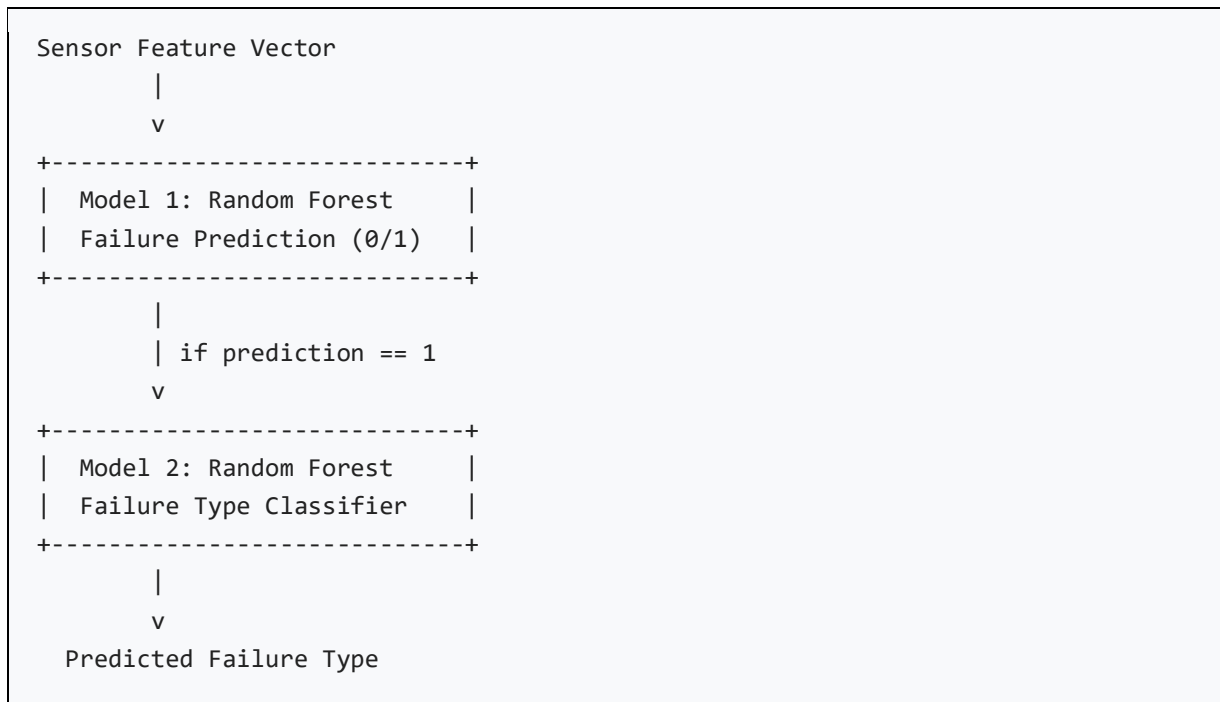
7.1 Stage 1 — Failure Prediction Model

This binary classifier consumes the full engineered feature set and outputs the probability that a given machine will experience a failure within the next 24 hours. A configurable probability threshold (default 0.5, tunable per machine type) converts this probability into a binary decision.

7.2 Stage 2 — Failure Type Classification Model

This multi-class classifier is invoked only when Stage 1 predicts an imminent failure. It reuses the same engineered feature set, augmented with the Stage 1 failure probability as an additional input feature, to classify the expected failure mode into one of: bearing, electrical, hydraulic, or motor_overheat.

Figure 2 — Two-Stage Model Architecture



7.3 Model Persistence

Both trained models, along with their corresponding encoders and feature scalers, are serialized using joblib and loaded once at FastAPI application startup to minimize inference latency.

8. FastAPI Deployment

FastAPI was selected as the model-serving framework due to its high performance, automatic OpenAPI documentation generation, native support for asynchronous request handling, and strong data validation through Pydantic schemas.

8.1 API Endpoints

Endpoint	Method	Description
/predict/failure	POST	Accepts sensor feature payload; returns failure_within_24h prediction and probability.
/predict/failure-type	POST	Accepts sensor feature payload; returns predicted failure_type and class probabilities.
/health	GET	Returns service health status for monitoring.
/model-info	GET	Returns model version, training date, and feature schema.

8.2 Sample Request / Response

Figure 3 — Sample /predict/failure Request

```
POST /predict/failure
{
  "machine_type": "Pump",
  "vibration_rms": 4.82,
  "temperature_motor": 78.3,
  "current_phase_avg": 12.4,
  "pressure_level": 6.1,
  "rpm": 1450,
  "operating_mode": "high_load",
  "hours_since_maintenance": 610,
  "ambient_temp": 31.2
}
```

Figure 4 — Sample Response

```
{
  "failure_within_24h": 1,
  "failure_probability": 0.87,
  "model_version": "rf_v1.2"
}
```

8.3 Public Exposure via ngrok

Since the FastAPI service runs in a local/development environment during the project phase, ngrok is used to expose the local server through a secure public HTTPS tunnel. This allows the n8n workflow (and any external client, such as Power BI or the Gemini AI agent) to reach the prediction endpoints without requiring a full cloud deployment, which is well-suited to an academic or portfolio-stage project.

In a production deployment, ngrok would be replaced with a persistent cloud-hosted API behind a proper domain, load balancer, and authentication layer.

9. n8n Workflow Design

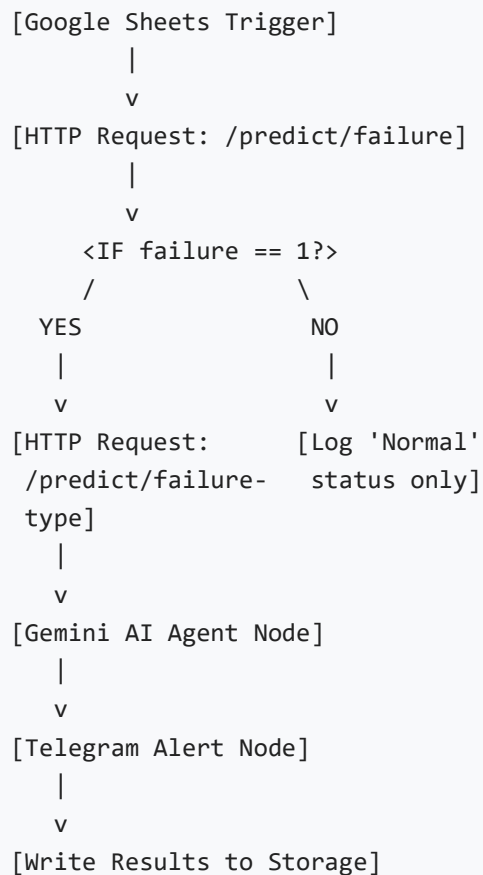
n8n serves as the orchestration backbone of the platform, connecting the data source, machine learning API, AI agent, notification system, and reporting layer into a single automated workflow, without requiring custom backend orchestration code.

9.1 Workflow Nodes

8. Google Sheets Trigger / Read Node — polls the source spreadsheet for new sensor rows on a scheduled interval.
9. HTTP Request Node (Failure Prediction) — sends the row's feature values to the FastAPI `/predict/failure` endpoint.
10. IF / Switch Node — branches the workflow based on whether `failure_within_24h` equals 1.
11. HTTP Request Node (Failure Type) — invoked only on the failure branch; calls `/predict/failure-type`.
12. AI Agent Node (Gemini) — generates a natural-language condition analysis and maintenance recommendation.
13. Telegram Node — sends the formatted alert message to the maintenance team's Telegram chat/group.
14. Google Sheets / Database Write Node — logs the prediction, failure type, and recommendation for downstream reporting.

9.2 Workflow Diagram

Figure 5 — n8n Automation Workflow



10. AI Agent Integration (Google Gemini)

A Google Gemini AI agent is embedded in the workflow to translate raw model outputs into actionable, human-readable guidance. Rather than presenting maintenance engineers with a bare probability score, the AI agent contextualizes the prediction using the underlying sensor readings, the predicted failure type, and general domain knowledge about industrial equipment.

10.1 Agent Responsibilities

- Summarize the machine's current condition in plain language.
- Explain which sensor readings most likely triggered the failure prediction.
- Generate a specific, prioritized maintenance recommendation aligned with the predicted failure_type.
- Estimate urgency level (e.g., immediate, within 24 hours, within one week) to help prioritize technician dispatch.

10.2 Sample Agent Output

Figure 6 — Example AI Agent Recommendation

Machine: Pump-07

Prediction: Failure likely within 24 hours

Failure Type: Hydraulic

Analysis: Pressure levels are 23% above the normal operating band while RPM remains stable, suggesting a developing hydraulic seal or line restriction issue.

Recommendation: Inspect hydraulic seals and pressure relief valve within the next 8 hours. Schedule a full hydraulic system check at next planned stoppage.

Urgency: HIGH

11. Telegram Notification System

Telegram was selected as the alerting channel due to its lightweight Bot API, reliable push delivery, and ease of integration with n8n through a native node, making it well suited for near-real-time maintenance notifications without requiring a dedicated mobile application.

11.1 Notification Content

- Machine identifier and machine type.
- Predicted failure status and probability.
- Predicted failure type (when applicable).
- AI-generated condition summary and recommendation.
- Timestamp of the triggering sensor reading.

11.2 Sample Alert Message

Figure 7 — Telegram Alert Format

```
ALERT: Predictive Maintenance System
Machine: CNC-14 | Type: CNC
Status: FAILURE PREDICTED (Probability: 0.91)
Failure Type: Bearing Wear
Recommendation: Replace spindle bearing within 12
hours. Reduce load to 60% until serviced.
Time: 2026-07-19 09:42
```

12. Power BI Dashboard

Power BI serves as the platform's reporting and business intelligence layer, connecting to the logged prediction results to provide plant managers and maintenance supervisors with a consolidated operational view.

12.1 Dashboard Components

- KPI cards summarizing total machines monitored, active alerts, and predicted failures in the last 24 hours.
- Failure trend line chart showing predicted failures over time, segmented by machine type.
- Failure type distribution chart (bearing, electrical, hydraulic, motor_overheat).
- Machine health heatmap highlighting machines with elevated failure probability.
- Maintenance response table listing recent alerts, recommendations, and resolution status.

12.2 Dashboard Layout Description

Figure 8 — Power BI Dashboard Layout

```
+-----+
| KPI: Machines | KPI: Active Alerts | KPI: Failures |
+-----+
```

	Failure Trend (Line Chart)		Failure Type (Pie)	
+	-----	+	-----	+
	Machine Health Heatmap (by machine_type & unit)			
+	-----	+	-----	+
	Recent Alerts & Recommendations (Table)			
+	-----	+	-----	+

13. Challenges Faced

- 15. Class imbalance: failure events are naturally rare compared to normal operating records, risking a model biased toward predicting 'no failure'.
- 16. Multi-stage latency: chaining two model inferences, an AI agent call, and a notification step introduced cumulative response time within the n8n workflow.
- 17. Data source limitations: using Google Sheets as a live data source is convenient for prototyping but introduces polling delays compared to a true streaming source.
- 18. Local API exposure: relying on ngrok for public access introduces a temporary tunnel URL that can change between sessions, requiring workflow reconfiguration.
- 19. Feature consistency: ensuring the exact same feature engineering steps applied during training were replicated at inference time inside the FastAPI service.

14. Solutions Implemented

- 20. Applied class-weighted Random Forest training and stratified train-test splitting to mitigate class imbalance and preserve failure-class recall.
- 21. Optimized the n8n workflow by branching early (IF node) so the failure-type model and AI agent are only invoked when Stage 1 predicts a failure, reducing unnecessary calls.
- 22. Documented and scheduled the Google Sheets polling interval to balance timeliness with API rate limits.
- 23. Established a reserved ngrok domain for the development phase to maintain a stable endpoint URL across sessions.
- 24. Packaged the feature engineering logic into a shared preprocessing module used identically by both the training pipeline and the FastAPI inference service, eliminating training-serving skew.

15.3 Key Observations

- The failure prediction model achieves strong recall, minimizing missed failure events, which is the priority metric for this use case.

-
- motor_overheat and 'none' classes are the easiest to classify correctly, likely due to distinctive temperature and vibration signatures.
 - electrical failures show the highest confusion with bearing failures, suggesting overlapping vibration signatures that could benefit from additional current-based features.
 - End-to-end latency from sensor read to Telegram alert delivery averaged under 5 seconds during workflow testing.

16. Future Improvements

- Replace the Google Sheets data source with a true streaming ingestion layer (e.g., MQTT broker or Kafka) for real industrial deployment.
- Deploy the FastAPI service on a persistent cloud platform (e.g., AWS, Azure, or GCP) with autoscaling, replacing ngrok for production use.
- Introduce model monitoring and drift detection to trigger automatic retraining as machine behavior evolves over time.
- Extend the failure prediction horizon beyond 24 hours with multiple prediction windows (e.g., 6h, 24h, 72h).
- Incorporate time-series/sequence models (e.g., LSTM or Temporal Convolutional Networks) to capture degradation trends rather than single-point-in-time snapshots.
- Add role-based access control and authentication to the FastAPI endpoints and Power BI workspace.
- Implement a feedback loop where maintenance technicians confirm actual failure outcomes to continuously improve model accuracy.

17. Conclusion

The Industrial Predictive Maintenance Platform demonstrates a complete, practical integration of data engineering and machine learning to solve a real industrial problem: anticipating equipment failure before it happens. By combining a two-stage Random Forest modeling architecture, a FastAPI serving layer, n8n-based workflow orchestration, Gemini AI-driven analysis, Telegram-based alerting, and Power BI reporting, the platform delivers an automated pipeline that spans the entire journey from raw sensor data to actionable maintenance decisions.

Beyond its technical implementation, this project illustrates the value of predictive maintenance as a business strategy, reduced downtime, lower costs, extended equipment life, and improved safety, while showcasing a technology stack and engineering approach directly transferable to real-world industrial AI systems. As such, it serves as a strong demonstration of applied data engineering and machine learning capability suitable for academic evaluation, professional portfolios, and technical interviews.

End of Document